Canonical pour Cubical Agda

Ewen DUFOUR

Supervisé par
Pierre Boutry (IGG, ICube, Université de Strasbourg)
Chase Norman (Carnegie Mellon University)
Loïc Pujet (IGG, ICube, Université de Strasbourg)

Mars - Août 2026

Contexte général

Les assistants de preuves sont des logiciels permettant de formaliser des résultats mathématiques et ainsi d'aider leurs utilisateurs à rédiger des preuves correctes. Certains assistants à la preuve, tels que Rocq [16], Lean [10] ou Agda [13], s'appuient sur la théorie des types dépendants pour représenter les énoncés mathématiques et leurs preuves. Au sein de ces systèmes, un énoncé est représenté par un type, tandis qu'une preuve est représentée par un terme habitant ce type. Le problème de l'habitation des types constitue donc un enjeu majeur dans le domaine de la formalisation automatisée.

Canonical [14] est un logiciel de recherche de preuve se basant sur une théorie des types dépendants dont le format est optimisé pour la recherche de termes. Son intégration dans l'assistant de preuve Lean a montré que cette approche s'intègre particulièrement bien aux assistants de preuves fondés sur la théorie des types dépendants. Elle présente notamment des performances supérieures à celles des autres logiciels de recherche de preuves disponibles dans Lean.

Problématique de recherche

La théorie cubique des types[5] étant une théorie des types de bas niveau, les preuves qui y sont réalisées requièrent parfois de nombreuses étapes administratives.

Il existe de nombreux logiciels de recherche de preuves automatisée reposant sur différentes théories des types. Cependant, il n'existe pour l'instant aucune intégration pleinement satisfaisante de ces logiciels dans la théorie cubique des types.

Ce travail a pour objectif de proposer une intégration du logiciel Canonical dans l'assistant de preuve Agda, en vue de son utilisation avec la théorie cubique des types.

Votre contribution

Dans le cadre du travail réalisé au cours de ce stage, nous proposons deux contributions principales.

Dans un premier temps, nous présentons une intégration de Canonical dans l'assistant à preuve Agda. Nous présentons d'abord Canonical (sous-section 2.1), ainsi qu'Agda (sous-section 2.2), avant

de donner un algorithme permettant de convertir des types d'Agda en types de Canonical (sous-section 2.4). Cette contribution s'accompagne d'un premier prototype [1], encore en cours de développement, permettant d'appeler directement Canonical depuis Agda.

Dans un second temps, nous présentons nos travaux concernant l'utilisation de Canonical pour la théorie cubique des types. Nous introduisons d'abord un encodage des chemins en théorie cubique des types (sous-section 3.1), ainsi que des contraintes qui leur sont associées (sous-section 3.2). Nous terminons par une discussion de l'intégration de l'opérateur de composition homogène dans notre prototype, ainsi que de nos résultats expérimentaux (sous-section 3.3).

Arguments en faveur de sa validité

Notre prototype reposant sur des branches de Canonical encore en cours de développement, et étant lui-même toujours en développement, aucune preuve formelle n'a encore été établie concernant les traductions entre les théories des types d'Agda et de Canonical.

Cependant, de nombreux tests ont été réalisés afin d'évaluer les différentes fonctionnalités de notre prototype. Ceux-ci ont notamment permis de mettre en évidence les limites de nos différentes approches pour la traduction des chemins cubiques et se sont révélés prometteurs quant à l'approche finale que nous présentons.

Sommaire et travaux futurs

Le travail réalisé au cours de ce stage constitue une preuve de concept pour l'automatisation de la recherche de preuves en théorie cubique des types. Nous y proposons deux contributions principales. La première consiste en une intégration partielle du logiciel de recherche de preuves Canonical au sein de l'assistant de preuves Agda. Nous nous sommes intéressés à la conversion des types d'Agda, représentés selon leur syntaxe interne, afin de pouvoir fournir des buts à Canonical via sa couche de compatibilité. Dans un second temps, nous nous sommes appuyés sur l'intégration présentée dans la première partie afin d'automatiser certaines preuves en théorie cubique des types. Cette automatisation repose notamment sur un encodage des types de chemins et des contraintes de cohérence associées à leurs extrémités.

Dans le cadre de travaux futurs, nous souhaiterions dans un premier temps améliorer l'utilisabilité de Canonical au sein d'Agda. Cela passerait par la possibilité de récupérer automatiquement les définitions de fonctions afin de les transformer en équations qui seraient ensuite fournies à Canonical, par la gestion du pattern matching d'Agda, ainsi que par différentes corrections de bugs et améliorations de l'interface utilisateur.

Dans un second temps, nous souhaiterions poursuivre l'étude des différentes manières d'encoder la théorie cubique des types et ses spécificités au sein d'autres théories des types, telles que celle utilisée par Canonical. Cette voie constitue l'une des pistes que nous explorerons dans le cadre de ma thèse, qui a pour objectif de rendre plus efficace l'évaluation des preuves en théorie homotopique et cubique des types.

1. Le prototype est disponible sur ce dépôt GitHub : https://github.com/Unistra-Preuves/agda.git

1 Contexte du stage

Dans cette section, nous présentons le contexte dans lequel ce stage a été réalisé. Nous nous intéressons à la théorie des types dépendants et à ses applications, ainsi qu'à la recherche de preuves automatisée. Nous terminons par une brève présentation de la théorie cubique des types.

1.1 La théorie des types dépendants.

Un système de preuves. Une théorie des types est un système formel dans lequel on s'intéresse notamment aux programmes bien typés. Celle-ci est définie par un ensemble de termes (programmes) constructibles, un ensemble de types constructibles, ainsi qu'un ensemble de règles de typage permettant d'attribuer des types aux termes et de définir les conversions entre types. Nous notons $\mathsf{t} : \mathsf{T}$ pour indiquer qu'un terme t est de type T et $\vdash \mathsf{t} : \mathsf{T}$ que le contexte permet de déduire que t est de type T.

Types

$$\tau ::= \iota \mid \tau \to \tau$$

Termes

$$e ::= x \mid \lambda x : \tau.\, e \mid e\, e$$

Contextes

$$\Gamma ::= \varnothing \mid \Gamma, x : \tau$$

$$\frac{x : \tau \in \Gamma}{\Gamma \vdash x : \tau}\ (\text{Var})$$

$$\frac{\Gamma, x : \tau_1 \vdash e : \tau_2}{\Gamma \vdash \lambda x : \tau_1.\, e : \tau_1 \to \tau_2}\ (\text{Abs})$$

$$\frac{\Gamma \vdash e_1 : \tau_1 \to \tau_2 \qquad \Gamma \vdash e_2 : \tau_1}{\Gamma \vdash e_1\, e_2 : \tau_2}\ (\text{App})$$

FIGURE 1 – Termes, types (gauche) et règles de typage (droite) du λ-calcul simplement typé

Correspondance de Curry-Howard. Dans ses travaux [11], Howard établit un lien profond entre les propositions de la logique intuitionniste et les types. Via cette correspondance, prouver qu'une formule A est vraie revient à exhiber un programme construisant un objet de type A. En enrichissant nos types en permettant aux types de dépendre de termes, on obtient un système de preuves dans lequel on peut décrire tous les énoncés mathématiques, la théorie des types dépendants [12].

Assistants à la preuve. Les assistants à la preuve comme Rocq, Lean et Agda sont basés sur des théories des types dépendants. Dans ces systèmes, on décrit un énoncé à prouver via un type, et une preuve de cet énoncé via un terme qui à le type correspondant. L'assistant vérifie que le terme possède le bon type avec un algorithme de type-checking [6]. Dans la figure 2, la première ligne permet de spécifier le type (l'énoncé) à prouver, et la deuxième décrit un terme (une preuve) de ce type dans la syntaxe d'Agda [2].

2. Le manuel de référence d'Agda : https://agda.readthedocs.io/en/latest/

```
modus-ponens : {A B : Prop} → (imp : A → B) → (a : A) → B
modus-ponens imp a = imp a
```

FIGURE 2 – Preuve du modus ponens en Agda

1.2 La théorie cubique des types

1.2.1 La théorie homotopique des types.

En mathématiques, on étudie souvent des objets à travers différentes représentations. Cependant, les propriétés intrinsèques de ces objets ne dépendent pas de la représentation choisie. Par exemple, un graphe peut être étudié à travers sa liste d'adjacence ou sa matrice d'adjacence.

C'est dans le but de formaliser cette indépendance vis-à-vis de la représentation que l'axiome d'univalence a été introduit par Voevodsky. Celui-ci stipule que, pour deux types, l'existence d'une équivalence entre ceux-ci est liée à l'existence d'une égalité entre ces deux types. Une équivalence entre deux types A et B est une fonction $\mathsf{f} : \mathsf{A} \to \mathsf{B}$ qui admet un inverse à gauche (une fonction $\mathsf{g} : \mathsf{B} \to \mathsf{A}$ avec $\mathsf{g} \circ \mathsf{f} = \mathsf{id}$) et un inverse à droite (une fonction $\mathsf{h} : \mathsf{B} \to \mathsf{A}$ avec $\mathsf{f} \circ \mathsf{h} = \mathsf{id}$)

Cet axiome constitue le principal ajout de la théorie homotopique des types [17] (HoTT). Celle-ci possède de nombreuses propriétés. Du point de vue du mathématicien, cette théorie se présente comme une alternative à la théorie des ensembles, tout en offrant des outils puissants pour développer l'homotopie synthétique. L'extensionnalité des fonctions et des propositions est également démontrable dans cette théorie.

Cependant, l'ajout de l'axiome d'univalence brisent les propriétés calculatoires de la théorie des types, car aucune règle de calcul ne lui est associée.

1.2.2 Une interprétation constructive de l'univalence.

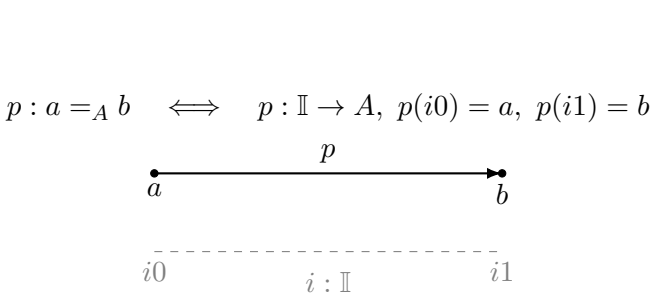
Dans le but de retrouver ces propriétés calculatoires, la théorie cubique des types [5] a été introduite par Cohen, Coquand, Huber et Mörtberg. Celle-ci ajoute un nouveau type I représentant l'intervalle $[0; 1]$ et donne une nouvelle interprétation de l'égalité. Une preuve d'égalité entre deux termes x et y d'un type A est interprétée comme une fonction de l'intervalle vers le type A qui, lorsqu'elle est évaluée en 0 (respectivement 1), doit être définitionnellement égale à x (respectivement y).

Picturalement, une égalité en théorie cubique des types peut être représentée comme une arête, une égalité entre égalités (une fonction de $\mathsf{I} \times \mathsf{I} \to \mathsf{A}$) comme un carré, et une fonction de $\mathsf{I} \times \mathsf{I} \times \mathsf{I} \to \mathsf{A}$ comme un cube, et ainsi de suite.

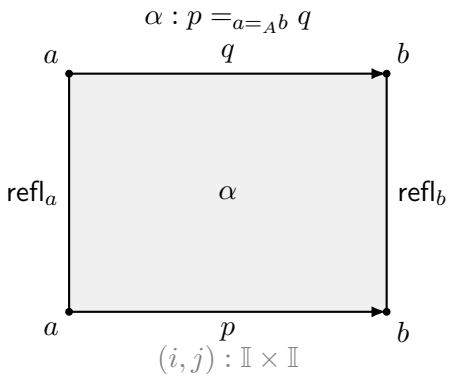
1.2.3 La composition homogène

Un autre ingrédient clé de la théorie cubique des types est l'opérateur de composition homogène. Introduit dans le CCHM [5], cet opérateur permet de compléter des cubes ouverts en dimension n. Pour cela, il suffit de lui spécifier un ensemble de faces cohérentes entre elles, ainsi qu'une base permettant de construire la face manquante.

Par exemple, dans la figure 4, les faces de gauche, de droite et du bas ont été fournies à l'opérateur hcomp, qui construit alors la face du haut. Comme mentionné précédemment, pour que cet opérateur fonctionne, les faces fournies doivent être cohérentes aux endroits où elles se



(a) Une égalité comme une arête



(b) Une égalité entre égalités comme un carré

FIGURE 3 – Interprétation géométrique des égalités en théorie cubique des types.

rencontrent. Supposons que $\mathsf{i} = \mathsf{i0}$ et $\mathsf{j} = \mathsf{i0}$, correspondant au coin en bas à gauche. Les deux arêtes qui se rencontrent doivent alors désigner un même terme, ici a.

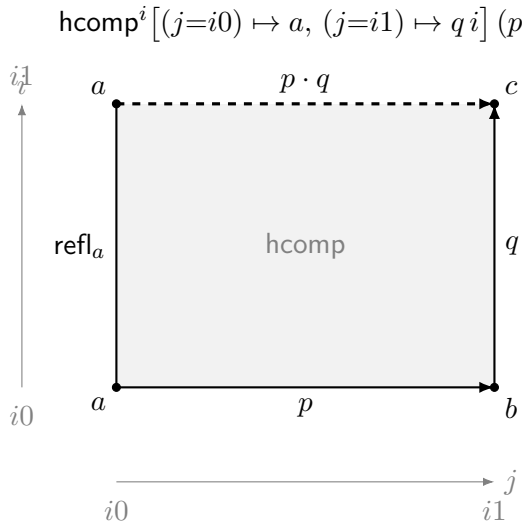


FIGURE 4 – Composition homogène : les faces de gauche, de droite et du bas sont fournies à hcomp, qui construit la face du haut. La face calculée est représentée en pointillés.

En dimension 3, cette contrainte de cohérence s'applique non plus à des coins, mais à des arêtes entières. En dimension 4, elle s'applique à des faces, et ainsi de suite.

La seule implémentation disponible sur un assistant de preuve répandu de la théorie cubique des types est celle fournie par Cubical Agda [18]. C'est pour cette raison que nous nous intéresserons à celui-ci dans le reste de ce travail.

1.3 La preuve automatisée en théorie cubique des types.

Un enjeu majeur de la théorie cubique des types est de parvenir à automatiser la recherche de preuves. Dans les théories des types dépendants plus classiques, il existe plusieurs classes d'outils permettant d'automatiser cette recherche.

Une première classe d'outils regroupe ceux qui cherchent à appliquer automatiquement des tactiques. On peut notamment citer Aesop [**?**] dans Lean, ainsi que les nombreuses tactiques de Rocq permettant d'effectuer ce type de recherche, telles que `tauto` ou `firstorder`.

Une deuxième classe de prouveurs automatiques traduit les obligations de preuve dans une logique du premier ordre. C'est notamment le cas de Coqhammer [7] dans Rocq et de Sledgehammer [3] dans Isabelle, qui transmettent ensuite les formules obtenues à des solveurs SMT ou des prouveurs automatiques tels que Vampire [2], CVC5 [1] ou Z3 [9].

Enfin, une dernière classe regroupe les logiciels reposant directement sur l'habitation de types. C'est notamment le cas d'Agsy et de son successeur Mimer [15] dans Agda.

À notre connaissance, aucune de ces approches n'est actuellement utilisée pour l'automatisation de la recherche de preuves en théorie cubique des types.

2 Traduction d'Agda vers Canonical

Dans cette section, nous présentons l'intégration de Canonical au sein d'Agda. Nous introduisons dans un premier temps Canonical, puis la solution étudiée pour permettre son utilisation depuis Agda. Nous terminons par la présentation de la méthode utilisée pour convertir les types d'Agda en types de Canonical.

2.1 Canonical

Canonical est un solveur pour les théories des types dépendants écrit en Rust par Norman et Avigad [14]. Celui-ci fonctionne sur sa propre théorie des types ce qui lui permet, entre autre, d'être totalement indépendant des assistants de preuves et de pouvoir potentiellement être utilisé avec plusieurs assistants. Il suffit de l'ajouter comme un plugin et de transformer les énoncés de l'assistant à la preuve en types de Canonical, les lui fournir pour qu'il effectue sa recherche de preuves et, en cas de succès, traduire le résultat dans le langage de l'assistant à la preuve. Dans leurs travaux [14], Norman et Avigad présentent une telle implémentation de Canonical dans l'assistant à la preuve Lean.

Fonctionnement. Étant donné un type, Canonical cherche à trouver un terme qui habite ce type en effectuant une recherche exhaustive en profondeur parmi les termes en forme normale. Le problème de l'habitation des types dans les théories des types dépendants étant semi-décidable, Canonical est ainsi garanti de trouver une solution au problème qui lui est soumis lorsqu'une telle solution existe.

Son implémentation dans Lean a montré des performances supérieures à celles d'Aesop et de Duper [4], deux autres outils de recherche de preuves. Nous avons donc choisi d'utiliser Canonical pour notre prototype, dans l'objectif de l'appliquer à la théorie cubique des types.

Théorie des types. Afin de restreindre au maximum son espace de recherche tout en conservant un niveau d'expressivité élevé, Canonical repose sur sa propre théorie des types, présentée dans la figure 5.

Celle-ci possède un unique constructeur de termes, composé d'une liste de paramètres, d'une liste de déclarations `let`, d'un symbole de tête et d'une liste d'arguments. Un type de Canonical est un terme possédant des annotations de type sur ses lieurs (`let` et λ).

Chaque terme de Canonical doit être sous forme β-normale, c'est-à-dire qu'aucune β-réduction ne doit être possible au sein du terme. De plus, tout symbole d'arité n doit être appliqué à exactement n arguments, quitte à η-étendre les fonctions. Le symbole de tête du terme doit être lié par un 'let' ou une λ-abstraction.

Au sein des types, chaque annotation de type est entièrement optionnelle. L'absence d'annotation de type pour un symbole indique à Canonical que celui-ci ne doit pas être utilisé pour construire le terme correspondant au but.

Types

$$X, Y, X', Y' ::= \text{let } (\bar{x} : \bar{X}) := \bar{M} \text{ in } \Pi(\bar{y} : \bar{Y}).\, g\, \bar{B}$$

Termes

$$A, B, C ::= \text{let } \bar{x} := \bar{M} \text{ in } \lambda \bar{y}.\, f\, \bar{A}$$

Contextes

$$\Gamma ::= \varnothing \mid \Gamma, x : \tau$$

Règles de typage des termes

$$\frac{f : \tau \in \Gamma}{\Gamma \vdash f : \tau}\ (\text{Symb})$$

$$\frac{\begin{array}{c}\Gamma, \bar{x} : \bar{X}, \bar{y} : \bar{Y} \vdash f : \text{let } (\bar{x'} : \bar{X'}) := \bar{M'} \text{ in } \Pi(\bar{y'} : \bar{Y'}).\, h\, \bar{C} \\ (h\, \bar{C})[\bar{A}[\bar{M}/\bar{x}]/\bar{y'}][\bar{M'}/\bar{x'}] =_\beta (g\, \bar{B})[\bar{M}/\bar{x}] \\ \forall i, \quad A_i[\bar{M}/\bar{x}] : Z_i[\bar{A}[\bar{M}/\bar{x}]/\bar{y'}][\bar{M'}/\bar{x'}]\end{array}}{\Gamma \vdash \text{let } \bar{x} := \bar{M} \text{ in } \lambda \bar{y}.\, f\, \bar{A} : \text{let } (\bar{x} : \bar{X}) := \bar{M} \text{ in } \Pi(\bar{y} : \bar{Y}).\, g\, \bar{B}}\ (\text{Tm})$$

FIGURE 5 – Grammaire et règles de typage des termes de Canonical.

Définitions, constantes. Canonical est capable de travailler avec des symboles représentant des constantes ainsi qu'avec des fonctions possédant une définition. Pour ce faire, il faut lui fournir, au moyen d'une déclaration `let`, le symbole ainsi que son type. Il est également possible de fournir à Canonical une définition associée à ce symbole, qui sont des règles de réécriture de la forme $\mathsf{f\ a_1\ a_2\ a_n = g\ b_1\ b_2\ b_m}$

Par exemple, supposons que l'on souhaite travailler avec la fonction $_ + _$ définie dans la figure 6. Il faut alors fournir à Canonical le type et la définition de $_ + _$. Cependant, le type de $_ + _$ faisant intervenir le type inductif des entiers naturels $\mathbb{N}$, ainsi que les symboles $\mathsf{Z} : \mathbb{N}$ et $\mathsf{S} : \mathbb{N} \to \mathbb{N}$, il est également nécessaire de fournir à Canonical la définition du type $\mathbb{N}$ ainsi que celles de ses symboles.

Le but aura donc la forme suivante :

$$
\begin{aligned}
&\mathit{let}\ldots,\\
&(\mathbb{N} : \mathit{Type}), (Z : \mathbb{N}), (S : \Pi(_ : \mathbb{N}).\mathbb{N}),\\
&(_ + _ : \Pi(_ : \mathbb{N}), (_ : \mathbb{N}).\mathbb{N})[\mathit{zero} + b = b; \mathit{suc}\ a + b = \mathit{suc}(a + b)],\\
&\ldots\mathit{in}\ T.
\end{aligned}
$$

```
data ℕ : Type where
  zero : ℕ
  suc : ℕ → ℕ

_+_ : ℕ → ℕ → ℕ
zero + b = b
suc a + b = suc (a + b)
```

FIGURE 6 – Définitions du type inductif des entiers naturel et de la fonction d'addition en Agda.

2.2 Agda

2.2.1 Fonctionnement d'Agda et appel à Canonical

Mode interactif. Au-delà de la vérification des preuves qui lui sont fournies, l'assistant à la preuve Agda peut être utilisé de manière interactive afin d'aider l'utilisateur à construire sa preuve.

Nous pouvons indiquer à Agda qu'une preuve est incomplète en laissant un ? au sein du terme de preuve, comme présenté dans la figure 7. Agda crée alors un nouveau but et lui associe le type que le terme manquant doit avoir pour compléter la preuve.

Par exemple, lors de la compilation du code présenté dans la figure 7, Agda crée le goal 0 présenté dans la figure 8. Les termes A et B étant implicitement introduits, tandis que le terme imp est explicitement introduit, Agda en déduit que le goal 0 doit être de type $A \to B$.

L'utilisateur peut alors se placer dans le goal 0 et y écrire un terme, qu'il fournira ensuite à Agda afin de compléter sa preuve.

```
modus-ponens : {A B : Prop} → (imp : A → B) → (a : A) → B
modus-ponens imp = ?
```

FIGURE 7 – Preuve incomplète du modus ponens en Agda

```
modus-ponens : {A B : Prop} → (imp : A → B) → (a : A) → B
modus-ponens imp = { }0
```

FIGURE 8 – Preuve incomplète compilée du modus ponens en Agda

Appel à Canonical. Dans le but d'aider l'utilisateur à compléter le but dans lequel il se trouve, nous souhaitons implémenter une commande permettant de récupérer le type du but courant et de le fournir à Canonical.

Deux solutions ont été envisagées afin de répondre à cette problématique. La première consiste à passer par le langage de réflexion d'Agda. Celui-ci permet d'écrire des tactiques de résolution directement sous la forme de termes d'Agda.

La deuxième solution consiste à implémenter une fonction permettant d'appeler Canonical directement dans le code Haskell d'Agda. Nous récupérons alors les types des buts dans la syntaxe interne des termes d'Agda puis, après une passe de transformation, transmettons le type du but à Canonical depuis notre prototype en Haskell vers l'implémentation en Rust de Canonical.

Comme nous souhaitons que notre prototype soit facilement utilisable, nous avons choisi de nous inspirer de la méthode employée, en partie, par Mimer et de nous concentrer sur la deuxième solution. Cela nous permet, à l'instar de la commande C-c C-a pour invoquer Mimer, d'invoquer Canonical via la commande C-c C-g du mode Agda de Emacs.

2.3 Cibles de la traduction

La théorie des types de Agda étant différente de celle de Lean, nous devons adapter les techniques présentées dans les travaux de Norman et Avigad [14] pour faire notre traduction.

2.3.1 La syntaxe interne d'Agda

Agda possède plusieurs syntaxes différentes pour représenter ses termes. Dans notre cas, nous nous intéressons à un sous-ensemble de la syntaxe dite interne des termes, dont une version simplifiée est présentée dans la figure 9.

Le sous-ensemble en question est décrit par quatre types mutuellement récursifs représentant respectivement les termes, les sortes, les types d'univers et les éliminateurs.

Les termes. Les termes sont composés de variables, représentées par des indices de De Bruijn [8], potentiellement appliquées à différents arguments, de λ-abstractions, ainsi que du type des fonctions dépendantes. Ils comprennent également des définitions, telles que les noms de fonctions ou de types inductifs, ainsi que des constructeurs, qui peuvent, eux aussi, être appliqués à différents arguments. Enfin, les termes peuvent contenir des sortes et des niveaux d'univers.

Dans la syntaxe réelle, les termes contiennent également d'autres constructions, telles que les littéraux ou les méta-variables, que nous n'aborderons pas dans ce travail.

Les éliminateurs. Au sein d'Agda, le type des éliminateurs permettent de représenter les applications de fonctions, les projections (types enregistrements) et les applications de chemin cubiques. Pour le moment, nous ne nous interessons qu'aux applications.

Les sortes et types d'univers. Pour ce stage, nous avons choisi de restreindre notre prototype aux seules sortes liées aux petits univers et à leurs niveaux.

Nous expliquons ce choix par le fait que la majorité des cas testés ne présentaient pas d'autres sortes que celles-ci. De plus, au vu de la gestion des univers par Canonical, que nous détaillerons par la suite, ajouter la prise en charge d'autres constructions ne ferait qu'ajouter du bruit à la recherche de termes.

```
data Term
  = Var Int [Elim]
  | Lam ArgInfo (Abs Term)
  | Pi (Dom Term) (Abs Term)
  | Def QName [Elim]
  | Con ConHead ConInfo [Elim]
  | Sort Sort
  | Level Level

data Elim
  = Apply (Arg Term)

data Sort
  = Univ Univ (Level Term)

data Univ
  = UType
```

FIGURE 9 – Sous ensemble simplifié de la syntaxe interne des termes d'Agda.

2.3.2 La cible.

Canonical est fourni avec une couche de compatibilité permettant de faire le lien avec le coeur du prover tout en gardant une syntaxe plus agréable à utiliser. La figure 10 montre la syntaxe pour représenter les termes et les types au sein de la couche de compatibilité en syntaxe Haskell.

CDecl. Ce type décrit les déclarations de la forme $(x : A)$. Il est composé d'un symbole, éventuellement d'un type, et d'équations.

Si le champ `typ` n'est pas renseigné, cela peut avoir deux significations.

Dans le cas d'une déclaration `let`, cela indique à Canonical que le symbole ne doit pas être utilisé pour construire le résultat. Dans le cas d'un paramètre, cela signifie que l'expression à laquelle la déclaration appartient représente un terme et non un type.

L'interprétation des équations associées aux déclarations dépend également du champ auquel ces dernières sont associées. Dans le cas d'une déclaration `let`, les équations représentent les règles de réduction associées au symbole (e.g. définition d'une fonction). Dans le cas d'un paramètre, elles représentent des contraintes de réduction sur ce paramètre.

CEquation. Ce type représente une règle de la forme $\mathsf{lhs} = \mathsf{rhs}$ à travers deux spines (un symbole de tête et une liste arguments). Par exemple, la déclaration associée à la fonction $\mathsf{not} : \mathsf{Bool} \to \mathsf{Bool}$, définie par $\mathsf{not}\ \mathsf{True} = \mathsf{False}$ et $\mathsf{not}\ \mathsf{False} = \mathsf{True}$, possèdera deux équations dont les membres de gauche seront CSpine "not" [trueExpr] (resp. CSpine "not" [falseExpr]) et les membres de droites seront CSpine "False" [] (resp. CSpine "True" []).

```
data CDecl = CDecl {
    name :: String,
    typ  :: Maybe CExpr,
    equations :: [CEquation]
  }
  deriving (Generic)

data CEquation = CEquation
  { lhs :: CSpine,
    rhs :: CSpine,
    is_redex :: Bool
  }
  deriving (Generic)

data CExpr = CExpr
  { params :: [CDecl],
    lets :: [CDecl],
    spine :: CSpine
  }
  deriving (Generic)

data CSpine = CSpine
  { head :: String,
    args :: [CExpr]
  }
  deriving (Generic)
```

FIGURE 10 – Syntaxe de la couche de compatibilité de Canonical.

CExpr. Ce type représente à la fois les termes et les types de Canonical que nous avons introduits dans la sous-section 2.1. La distinction entre les termes et les types s'effectue au niveau des champs `params` et `lets`. Contrairement aux types, un terme ne possédera pas d'annotations de type dans ces champs.

CSpine. Les spines sont composées d'un symbole de tête et d'une liste d'arguments. Elles représentent le corps des types et des termes de Canonical.

2.4 La traduction.

Maintenant que nous avons présenté les langages source et cible, intéressons-nous à la traduction elle-même. Notre traduction se décompose en trois fonctions mutuellement récursives permettant respectivement de transformer des termes en déclarations `toCDecl`, en expressions `toCExpr` ou en spines `toCSpine`.

Le point d'entrée. Canonical attend une déclaration en entrée pour effectuer son traitement. Notre point d'entrée est donc la fonction `toCDecl`, dont l'implémentation en Haskell est présentée en figure 11. Celle-ci prend simplement en arguments un nom, un terme à convertir et un ensemble d'équations. Elle appelle ensuite la fonction `toCExpr` sur le terme à convertir afin de construire la structure `CDecl` correspondante. Les autres arguments de la fonction sont utilisés dans des processus que nous détaillerons par la suite.

```
toCDecl :: Term -> -- ^ Le terme à convertir
           String -> -- ^ Le nom associé à la déclaration
           [CEquation] -> -- ^ Les contraintes / définitions
           [String] -> -- ^ Nom liés
           [CDecl] -> -- ^ Contexte
           Map String Bool -> -- ^ Définitions déjà rencontrées
           Bool -> -- ^ En train de transformer la déclaration "Goal"?
           TCM (CDecl, [CDecl], Map String Bool)
toCDecl t name eqs bindnames lets ald tplvl = do
  toCExpr t bindnames [] lets ald tplvl True
```

FIGURE 11 – Traduction des termes en CDecl.

2.4.1 Conversion en termes de Canonical

La fonction `toCExpr` a pour objectif principal de construire la liste des paramètres présents dans un terme ou un type de Canonical. Pour ce faire, la fonction parcourt le terme en conservant les paramètres déjà créés jusqu'à ce qu'elle rencontre un terme qui n'en crée pas de nouveau.

Termes créant des paramètres. Deux constructeurs se traduisent en termes de Canonical : `Lam` et `Pi`. Dans le cas de `Lam`, il suffit d'ajouter à la liste des paramètres une déclaration contenant uniquement le nom de la variable, puis de s'appeler récursivement sur le terme sous-jacent. Dans le cas de `Pi`, deux situations doivent être distinguées. Si nous sommes en train de convertir le terme en un type, il suffit de convertir le domaine du `Pi` en une `CDecl`, de l'ajouter aux paramètres, puis d'appeler récursivement `toCExpr` sur le codomaine. Si nous sommes en train de convertir le terme en un terme, il faut le traduire selon le procédé décrit dans la section 4.1 de [14]. Cette fonctionnalité n'a pas encore été implémentée dans notre prototype. La distinction entre la traduction vers un terme ou vers un type s'effectue en fonction de la fonction appelant `toCExpr`. Si celle-ci est appelée par `toCDecl`, cela signifie que nous sommes en train de convertir un type. Dans le cas contraire, `toCExpr` a été appelée par `toCSpine` et nous sommes en train de convertir les arguments d'une fonction, qui sont donc des termes.

Autres constructeurs. Dans le cas ou `toCExpr` est appelé sur d'autre constructeur que `Pi` et `Lam`, cela signifie que tous les paramètres ont été dépliés. Il faut donc convertir le termes vers une spine (constructeur de tête et arguments) et créer la `CExpr` en mettant les paramètres avec la spine.

Un point que nous n'avons pas encore abordé concerne la gestion du contexte du but. Celui-ci sert à mettre tout ce qui est en lien avec les constantes, les fonctions externes, les datatypes et les constructeurs. Notre stratégie consiste à mettre tout cet environnement sur l'expression

correspondant au type du goal. L'environnement se construisant tout au long de la conversion du terme, nous devons le garder en mémoire et l'ajouter qu'à la toute fin de la traduction, après avoir converti la spine du goal. Nous détaillerons la construction de l'environnement par la suite.

2.4.2 Conversion en spine.

La fonction `toCSpine` sert à convertir les termes de Agda vers un symboles de tête et ses potentiels arguments. Étant seulement appellée par la fonction `toCExpr` après avoir déplié tous les `Pi` et `Lam`, il nous suffit de seulement traiter les variables, les sortes, les niveaux d'univers, les constructeurs et les définitions. Pour rappel, les `CSpine` sont constitués d'un symbole de tête (représenté par une chaîne de caractères) et d'une liste d'arguments (liste de `CExpr`).

Les variables. Dans la syntaxe interne de Agda, une variable est représentée par un indice de De Bruijn et une liste d'éliminateurs. Ayant restreint les éliminateurs aux applications pour le moment, nous obtenons un terme déjà très proche d'une spine. Il nous suffit donc de convertir la liste d'éliminateurs en transformant chaque terme de la liste en `CExpr`, puis de construire la spine en mettant le nom associé à l'indice de De Bruijn avec les arguments convertis.

Les définitions et constructeurs. Les termes `Def` correspondent aux symboles étant rattachés à une définition comme les axiomes (introduits avec le mot clé *postulate* en Agda), les fonctions définies par l'utilisateur, les fonctions primitives ou encore les datatypes. Pour le moment notre prototype ne gère pleinement que les cas ou la définition correspond à un datatype ou un constructeur, cependant nous pensons que la technique présentée peut-être facilement adaptée pour gérer les autres cas.

Lorsque l'en rencontre un symbole correspondant à un datatype, on récupère le terme correspondant au type de ce datatype et on ajoute au contexte une déclaration contenant le symbole et le terme converti en CExpr. On récupère par la suite la liste des constructeurs associés à ce datatype et on les ajoute également au contexte via un procédé similaire.

Si on voulait convertir un symbole possédant une définition (comme une fonction définie par l'utilisateur), il suffirait de récupérer les différentes clauses de cette fonction et de les convertir en `CEquations`. N'ayant pas effectué cette implémentation pour le moment, nous ne détaillerons pas plus ce procédé.

Les univers et leurs niveaux. Comme indiqué dans [14], Canonical ne gère pas les niveaux d'univers. Nous fournissons donc simplement à Canonical un objet **Type** ne possédant pas de type dans son contexte. De cette manière, Canonical n'essaiera pas de typer les éléments de type `Type` et n'utiliseras pas `Type` pour construire ses termes.

3 Cubical Agda

Maintenant que nous avons un prototype pour les fonctions basiques de Agda, nous nous intéressons à son application pour la théorie cubique des types.

Le type intervalle. La première étape vers l'utilisation de Canonical pour la théorie cubique des types est le support de l'algèbre de De Morgan libre de l'intervalle. Nous ajoutons au contexte que

nous fournissons à Canonical trois déclarations (I : _), (i0 : I) et (i1 : I), représentant respectivement le type intervalle, l'élément `0` et l'élément `1`.

Nous ajoutons également des déclarations pour les fonctions suivantes :

— (primINeg : I → I)
  — primINeg i0 = i1
  — primINeg i1 = i0
  — primINeg (primIMax i j) = primIMin (primINeg i) (primINeg j)
  — primINeg (primIMin i j) = primIMax (primINeg i)(primINeg j)
— (primIMin : I → I → I)
  — primIMin i0 j = i0
  — primIMin i1 j = j
  — primIMin j i0 = i0
  — primIMin j i1 = j
— (primIMax : I → I → I)
  — primIMax i0 j = j
  — primIMax i1 j = i1
  — primIMax j i0 = j
  — primIMax j i1 = i1

Comme leurs noms l'indiquent, ces fonctions représentent la négation, le max et le min sur l'intervalle. À présent que nous avons une représentation du type intervalle, nous nous intéressons à la gestion des types d'égalité (aussi appelé path type dans CTT).

3.1 Représentation des path types

La première voie que nous avons explorée pour représenter les path type passe par l'utilisation d'un constructeur et d'un destructeur pour les path type. Pour ce faire, nous ajoutons au contexte de Canonical un constructeur

$$\mathsf{makePath} : (\ell : \mathit{Level}) \to (A : \mathit{Type}\ \ell) \to (x : A) \to (y : A) \to (p : I \to A)\{p\, i0 = x; p\, i1 = y\} \to x \equiv y$$

ainsi qu'un destructeur

$$\mathsf{destructPath} : (\ell : \mathit{Level}) \to (A : \mathit{Type}\ \ell) \to (x : A) \to (y : A) \to (p : x \equiv y) \to (i : I) \to A$$

avec les règles de réduction :

$$\mathsf{destructPath}\ \ell\ A\ x\ y\ p\ i0 = x,\ \mathsf{destructPath}\ \ell\ A\ x\ y\ p\ i1 \quad = y.$$

Le constructeur sert à contraindre Canonical à fournir une fonction p : I → A ayant les bonnes extrémités comme témoin pour justifier une égalité, tandis que le destructeur sert à imiter le comportement d'un path type.

Bien que cette approche fonctionne sur des cas simples, celle-ci a rapidement montré ses limites face à des obligations de preuve plus complexes. En plus d'alourdir fortement les preuves produites par Canonical, l'ajout des constructeurs et destructeurs augmente la taille de l'arbre de recherche de termes de Canonical. Nous avons donc choisi d'adopter une représentation plus proche de l'utilisation réelle des types de chemins.

La deuxième solution que nous avons adoptée consiste à remplacer toutes les occurrences du type x ≡ y par le type I → A, et à utiliser les contraintes natives de Canonical pour spécifier le

comportement de la fonction aux extrémités de l'intervalle. Par exemple, un objet $(p : x \equiv y)$ devient une fonction $(p : I \to A)$ accompagnée des contraintes $p\ i0 = x$ et $p\ i1 = y$.

3.2 Contraintes de cohérence.

Lorsqu'on travaille en théorie cubique des types, un enjeu majeur réside dans la gestion des contraintes imposées aux termes sur les extrémités de l'intervalle.

Par exemple, intéressons-nous à la preuve de l'extensionnalité des fonctions. Cet énoncé, prouvable en théorie cubique des types, stipule que deux fonctions égales point par point sont égales au sein du type des fonctions.

Pour prouver une égalité entre deux fonctions $(f\ g : A \to B)$ en CTT, il faut fournir une fonction $(h : I \to (A \to B))$ telle que $h\ i0 = f$ et $h\ i1 = g$. En introduisant une variable $(a : A)$, cela revient à imposer les contraintes $h\ i0\ a = f\ a$ et $h\ i1\ a = g\ a$.

Nous présentons en figure 12 un type décrivant l'extensionnalité des fonctions en Agda. Comme toutes les variables ont été introduites, le type du but est simplement B. Si l'on fournit tel quel le but à Canonical, celui-ci n'a aucune connaissance des contraintes que nous venons d'évoquer.

```
fe : (A B : Type) → (f g : A → B) → (p : (a : A) → f a ≡ g a) → f ≡ g
fe A B f g p i a = { }0
```

FIGURE 12 – Définition du type de l'extensionnalité des fonctions.

Pour palier ce problème, nous nous servons directement d'une fonctionnalité d'Agda. Dans un tel but, Agda calcule les contraintes que nous avons mentionnées et les affiches à l'utilisateur. En reprenant l'exemple de la figure 12, Agda affiche à l'utilisateur $(i = i0) \vdash f\ a$ et $(i = i1) \vdash g\ a$. En l'état, il n'est pas possible de spécifier à Canonical de telles contraintes. La solution que nous avons adoptée est de replier sur le but les éléments du contexte courant jusqu'à la variable d'intervalle qui nous intéresse et à spécifier les contraintes directement sur la déclaration du but. Le but 0 de la figure 12 passerais de $(Goal : B)$ à $(Goal : I \to A \to B)$ avec les contraintes $Goal\ i0\ a = f\ a$ et $Goal\ i1\ a = g\ a$. Pour revenir au but initial, il suffit de re-déplier le terme fourni par Canonical en se débarrassant des abstractions que nous avons ajouté.

3.3 La composition homogène avec Canonical

Comme mentionné lors de notre présentation des la théorie cubique des types, la composition homogène est un élément central pour la construction d'égalités. Nous souhaitons fournir à Canonical la possibilité de créer des chemins avec la composition homogène. Notre première approche était de fournir à Canonical toute la signature de l'implémentation de cet opérateur dans Agda, qui est décrit en section 3.3 et 3.4 de [18].

Nous avons testé cette approche pour prouver la transitivité des chemins et Canonical n'a fourni un résultat qu'au bout de plusieurs minutes. De plus, la cohérence des faces n'était pas respectée dans le résultat qu'il a fourni. Cela s'explique par le fait qu'Agda fasse ses vérifications en interne et pas seulement au niveau du type-checker, la signature seule de l'implémentation n'était donc pas suffisante.

Solution partielle. Pour le moment, nous n'avons pas trouvé de moyen de fournir à Canonical la signature complète de `hcomp`, accompagnée des contraintes sur ses faces. Nous avons donc essayé d'utiliser Canonical non plus pour trouver l'intégralité de la solution, mais comme une aide pour déterminer les faces respectant ces contraintes. Prenons l'exemple présenté en figure 13. Ici, les buts `0` et `1` doivent contenir les faces gauche et droite que nous avons décrites en figure 4. En appliquant la technique de repli dont nous avons discuté dans la section 3.2, nous parvenons à fournir à Canonical l'ensemble des contraintes dont il a besoin, et celui-ci fournit alors un résultat satisfaisant.

Par exemple, le but `1` à comme contraintes $\mathsf{j} = \mathsf{i0} \vdash \mathsf{y}$ et $\mathsf{j} = \mathsf{i1} \vdash \mathsf{z}$ et comme type `A`. En repliant `j` sur le goal, on obtient comme but $(\mathsf{j} : \mathsf{I}) \to \mathsf{A}$ avec comme contraintes $\mathsf{Goal\ i0} = \mathsf{y}$ et $\mathsf{Goal\ i1} = \mathsf{z}$. Canonical nous fourni donc le terme `q j`.

```
trans : (ℓ : Level) → (A : Type ℓ) → (x y z : A) → (p : x ≡ y) → (q : y ≡ z) → x ≡ z
trans ℓ A x y z p q i =
  hcomp {ℓ} {A} {~ i ∨ i} (λ j → λ {(i = i0) → { }0
                                   ;(i = i1) → { }1}) (p i)
```

FIGURE 13 – Preuve incomplète de la transitivité des chemins en Cubical Agda

Limites de l'approche. Bien que cette approche fonctionne pour des cas simples, comme celui de la transitivité, elle se retrouve bloquée lorsque les cas deviennent plus complexes. Le cas de la transitivité fonctionne bien, car fournir tous les arguments implicites ainsi que la face inférieure à `hcomp` permet à Agda de déduire l'ensemble des contraintes correspondant aux faces qui nous manquent. Cependant, si la contrainte associée à une face dépend d'une autre face qui n'a pas encore été spécifiée, Agda ne peut pas déduire cette contrainte et laisse une méta-variable à la place. En figure 14, les buts `0` et `1` sont interdépendants, et notre approche ne permet donc pas de traiter ce cas.

Résultats. La ou Mimer n'était pas en capacité de fournir de preuves utilisant la théorie cubique des types, notre approche a permis d'automatiser plusieurs preuves simples comme celles des buts présent en figures 12 et 13. Nous pensons que l'utilisation de Canonical présente un réel potentiel pour l'automatisation de preuves au sein de la théorie cubique des types.

4 Conclusion et travaux futurs

Nous rappelons ici ce qui est présenté dans la sous-section éponyme présente au début de ce rapport.

Le travail réalisé au cours de ce stage constitue une preuve de concept pour l'automatisation de la recherche de preuves en théorie cubique des types. Nous y proposons deux contributions principales. La première consiste en une intégration partielle du logiciel de recherche de preuves Canonical au sein de l'assistant de preuves Agda. Nous nous sommes intéressés à la conversion des types d'Agda, représentés selon leur syntaxe interne, afin de pouvoir fournir des buts à Canonical via sa couche de compatibilité. Dans un second temps, nous nous sommes appuyés sur l'intégration présentée dans la première partie afin d'automatiser certaines preuves en théorie cubique des types.

```
Square : {ℓ : Level} {A : Type ℓ} {w x y z : A}
       → (u : w ≡ x) (v : y ≡ z)
       → (r : w ≡ y) (s : x ≡ z)
       → Type ℓ
Square u v r s = PathP (λ i → r i ≡ s i) u v

invSides-filler : {ℓ : Level} {A : Type ℓ} {x y z : A}
                  (p : x ≡ y) (q : x ≡ z) → Square p (inv q) q (inv p)
invSides-filler {x = x} p q i j =
  hcomp (λ k → λ { (i = i0) → { }0
                 ; (i = i1) → q ((~ j) ∧ k)
                 ; (j = i0) → { }1
                 ; (j = i1) → p ((~ j) ∧ k)})
        x
```

FIGURE 14 – Preuve incomplète en Cubical Agda

Cette automatisation repose sur un encodage des types de chemins et des contraintes de cohérence associées à leurs extrémités.

Dans le cadre de travaux futurs, nous souhaiterions dans un premier temps améliorer l'utilisabilité de Canonical au sein d'Agda. Cela passerait notamment par la possibilité de récupérer automatiquement les définitions de fonctions afin de les transformer en équations qui seraient ensuite fournies à Canonical, par la gestion du pattern matching d'Agda, ainsi que par différentes corrections de bugs et améliorations de l'interface utilisateur.

Dans un second temps, nous souhaiterions poursuivre l'étude des différentes manières d'encoder la théorie cubique des types et ses spécificités au sein d'autres théories des types, telles que celle utilisée par Canonical. Cette voie constitue l'une des pistes que nous explorerons dans le cadre de ma thèse, qui a pour objectif de rendre plus efficace l'évaluation des preuves en théorie homotopique et cubique des types.

Remerciements

Je tiens à remercier tes chaleureusement Pierre Boutry et Loïc Pujet pour toutes les opportunités qu'ils m'ont offerts ainsi que pour toute l'aide qu'il m'ont apporté durant ce stage et pour mes candidatures a des bourses doctorales.

Je suis également reconnaisant envers Chase Norman pour son aide précieuse et ses explications concernant le fonctionnement Canonical.

J'aimerais remercier Aymeric Feltz, Damien Mariotti et Lyes Saadi avec qui j'ai passé une grande partie de mon stage et qui m'ont bien souvent servi de canards en plastique.

Enfin, je souhaite remercier l'équipe IGG qui s'est montrée particulièrement accueuillante tout au long de ce stage.

Références

[1] H. Barbosa, C. Barrett, M. Brain, G. Kremer, H. Lachnitt, M. Mann, A. Mohamed, M. Mohamed, A. Niemetz, A. Nötzli, et al. cvc5 : A versatile and industrial-strength smt solver. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 415–442. Springer, 2022.

[2] F. Bártek, A. Bhayat, R. Coutelier, M. Hajdú, M. Hetzenberger, P. Hozzová, L. Kovács, J. Rath, M. Rawson, G. Reger, et al. The vampire diary. In *International Conference on Computer Aided Verification*, pages 57–71. Springer, 2025.

[3] S. Böhme and T. Nipkow. Sledgehammer : judgement day. In *International Joint Conference on Automated Reasoning*, pages 107–121. Springer, 2010.

[4] J. Clune, Y. Qian, A. Bentkamp, and J. Avigad. Duper : a proof-producing superposition theorem prover for dependent type theory. In *15th International Conference on Interactive Theorem Proving (ITP 2024)*, pages 10–1. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, 2024.

[5] C. Cohen, T. Coquand, S. Huber, and A. Mörtberg. Cubical type theory : a constructive interpretation of the univalence axiom. *arXiv preprint arXiv :1611.02108*, 2016.

[6] T. Coquand. An algorithm for type-checking dependent types. *Science of computer programming*, 26(1-3) :167–177, 1996.

[7] Ł. Czajka and C. Kaliszyk. Hammer for coq : Automation for dependent type theory. *Journal of automated reasoning*, 61(1) :423–453, 2018.

[8] N. G. De Bruijn. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the church-rosser theorem. In *Indagationes mathematicae (proceedings)*, volume 75, pages 381–392. Elsevier, 1972.

[9] L. De Moura and N. Bjørner. Z3 : An efficient smt solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 337–340. Springer, 2008.

[10] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, pages 625–635, 2021.

[11] W. A. Howard. The formulae-type notion of construction. *To H.B. Curry : essays on combinatory logic, lambda calculus and formalism*, pages 479–490, 1980.

[12] P. Martin-Löf. *Intuitionistic Type Theory*, volume 1 of *Studies in Proof Theory*. Bibliopolis, Naples, 1984. Notes by Giovanni Sambin.

[13] U. Norell. Dependently typed programming in Agda. In *International School on Advanced Functional Programming (AFP)*, 2008.

[14] C. Norman and J. Avigad. Canonical for automated theorem proving in lean. *arXiv preprint arXiv :2504.06239*, 2025.

[15] L. Skystedt. A new synthesis tool for agda. 2022.

[16] The Rocq Development Team. The Rocq prover, 2025.

[17] The Univalent Foundations Program. *Homotopy Type Theory : Univalent Foundations of Mathematics*. Institute for Advanced Study, Princeton, 2013.

[18] A. Vezzosi, A. Mörtberg, and A. Abel. Cubical agda : a dependently typed programming language with univalence and higher inductive types. *Proceedings of the ACM on Programming Languages*, 3(ICFP) :1–29, 2019.

A Sommaire

Table des matières

1 Contexte du stage **3**
- 1.1 La théorie des types dépendants. . . . 3
- 1.2 La théorie cubique des types 4
  - 1.2.1 La théorie homotopique des types. . . . 4
  - 1.2.2 Une interprétation constructive de l'univalence. . . . 4
  - 1.2.3 La composition homogène 4
- 1.3 La preuve automatisée en théorie cubique des types. . . . 6

2 Traduction d'Agda vers Canonical **6**
- 2.1 Canonical 6
- 2.2 Agda 8
  - 2.2.1 Fonctionnement d'Agda et appel à Canonical 8
- 2.3 Cibles de la traduction 9
  - 2.3.1 La syntaxe interne d'Agda 9
  - 2.3.2 La cible. . . . 10
- 2.4 La traduction. . . . 11
  - 2.4.1 Conversion en termes de Canonical 12
  - 2.4.2 Conversion en spine. . . . 13

3 Cubical Agda **13**
- 3.1 Représentation des path types 14
- 3.2 Contraintes de cohérence. . . . 15
- 3.3 La composition homogène avec Canonical 15

4 Conclusion et travaux futurs **16**

A Sommaire **19**

B Code source complémentaire **20**

B Code source complémentaire

```
toCExpr :: Term ->           -- ^ Le terme a convertir
           [String] ->       -- ^ Variables liees
           [CDecl] ->        -- ^ Parametres
           [CDecl] ->        -- ^ Contexte
           Map String Bool -> -- ^ Définitions déjà rencontrées
           Bool ->           -- ^ En train de convertir le Goal ?
           Bool ->           -- ^ En train de convertir vers un type ?
           TCM (CExpr, [CDecl], Map String Bool)
toCExpr t bindnames pidecl letdecl ald tplvl totyp =
  case t of
    Pi a b -> do
      (newnames, na) <- do case b of
                           NoAbs _ _ -> return (bindnames, "_")
                           Abs n _ -> do
                             n' <- freshString n
                             return (n' : bindnames, n')
      (domdecl, lets, ald) <- toCDecl (unEl $ unDom a) na [] bindnames letdecl ald False
      toCExpr (unEl $ unAbs b) newnames (domdecl : pidecl) lets ald tplvl totyp
    Lam a b -> do
      let (newnames, name) = case b of
                    NoAbs _ _ -> (bindnames, "_")
                    Abs n _ -> (n : bindnames, n)
      toCExpr (unAbs b) newnames (CDecl name Nothing [] : pidecl ) letdecl ald tplvl False
    _ -> do
      (spine, lets, ald) <- toCSpine t bindnames letdecl ald
      return $ (
        CExpr {
            params = reverse pidecl,
            lets = if tplvl then reverse lets else [],
            spine
          }, lets, ald)
```

FIGURE 15 – Traduction des termes en `CExpr`.

```
toCSpine :: Term ->           -- ^ Le termes à convertir
            [String] ->       -- ^ Les variables liées
            [CDecl] ->        -- ^ Contexte
            Map String Bool -> -- ^ Définitions déjà rencontrées
            TCM (CSpine, [CDecl], Map String Bool)
toCSpine t bindnames lets ald =
  case t of
    Var i el -> do
      (args, lets, ald) <- elimsToCExpr el bindnames lets ald
      return $ ( CSpine (bindnames !! i) args, lets, ald)
    Sort (Type l) -> do
      t' <- reallyUnLevelView l
      (arg, lets, ald) <- toCExpr t' bindnames [] lets ald False False
      return $ (
        CSpine {
          head = "Type",
          args = [arg]
        }, lets, ald)
    Level l -> do
      t' <- reallyUnLevelView l
      toCSpine t' bindnames lets ald
    Def qname e -> do
      (args, lets, ald) <- elimsToCExpr e bindnames lets ald
      (lets, ald) <- gatherDatatypeInformations qname bindnames lets ald
      return $ (
        CSpine {
          head = P.prettyShow (qnameName qname),
          args
        }, lets, ald)
    Con hd _ e -> do
      infos <- getConstInfo (conName hd)
      let dataname = case theDef infos of
                   ConstructorDefn cd -> _conData cd
                   _ -> __IMPOSSIBLE__
      (args, lets, ald) <- elimsToCExpr e bindnames lets ald
      (lets, ald) <- gatherDatatypeInformations dataname bindnames lets ald
      return $ (
        CSpine {
          head = P.prettyShow . qnameName $ conName hd,
          args
        }, lets, ald)
```

FIGURE 16 – Fonction de conversion des termes en `CSpine`.